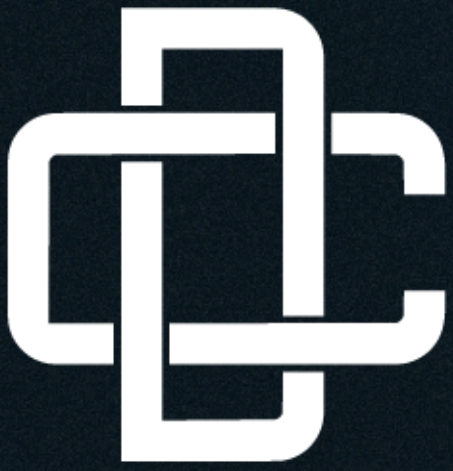

CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

BuilDefi

March 2026

Prepared by

ZanyBonzy

Varun_Sharma

radev_eth

Introduction

A time-boxed security review of the **BuilDefi** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About **BuilDefi**

BuilDefi is a token launch and DeFi infrastructure protocol built on top of Uniswap V4. Projects launch tokens through a fair launch mechanism where participants mint tokens by sending ETH and once the launch window closes the collected ETH is distributed across liquidity pools, creator allocations, buy-and-burn modules and protocol fees. Launched tokens are tied to an ownership NFT that can change hands through a community takeover (CTO) mechanism and generate ongoing trading fees that flow through a leaderboard system rewarding the most actively traded tokens each epoch. The protocol also includes optional staking vaults, Aave-based external staking for yield generation, a points and boost system for user engagement and a whitelist registry managing which external tokens can be used as LP pair assets.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

***review commit hash* - 833aee1d0057195c8253502271a5f0a62ef67f4f**

***fixes review commit hash* - 41d720e6cdf92071b822ed39c4413b3773bb59bc**

Scope

The following smart contracts were in scope of the audit:

- BDeFiAaveStaking.sol
- BDeFiBuyBurn.sol
- BDeFiStaking.sol
- BDeFiToken.sol
- BDeFiConfigNames.sol
- BDeFiContractNames.sol
- BDeFiBoost.sol
- BDeFiCloner.sol
- BDeFiConfig.sol
- BDeFiCore.sol
- BDeFiHook.sol
- BDeFiLaunch.sol
- BDeFiLeaderboard.sol
- BDeFiLpManager.sol
- BDeFiLpManagerActions.sol
- BDeFiNft.sol
- BDeFiPayments.sol
- BDeFiPayouts.sol
- BDeFiPoints.sol
- BDeFiSwapHandler.sol
- BDeFiTaxDistributor.sol
- BDeFiWhitelist.sol
- IBDeFiAaveStaking.sol
- IBDeFiBoost.sol
- IBDeFiBuyBurn.sol
- IBDeFiCloner.sol
- IBDeFiConfig.sol
- IBDeFiCore.sol
- IBDeFiHook.sol
- IBDeFiLaunch.sol
- IBDeFiLeaderboard.sol
- IBDeFiLpManager.sol
- IBDeFiLpManagerTypes.sol
- IBDeFiNft.sol
- IBDeFiPayments.sol
- IBDeFiPayouts.sol
- IBDeFiPoints.sol
- IBDeFiStaking.sol
- IBDeFiSwapHandler.sol
- IBDeFiTaxDistributor.sol
- IBDeFiToken.sol
- IBDeFiWhitelist.sol
- IERC20Burnable.sol

- ITruncatedGeoOracle.sol
- IWETH.sol
- IAToken.sol
- IAavePool.sol
- IPoolAddressesProvider.sol
- IPoolDataProvider.sol
- IRewardsController.sol
- IScaledBalanceToken.sol
- AggregatorV3Interface.sol
- AllocPacking.sol
- DuplicationCheck.sol
- HookLibrary.sol
- LpManagerLibrary.sol
- RouteLibrary.sol
- StringValidator.sol
- MathUtils.sol
- WadRayMath.sol
- TruncatedGeoOracle.sol
- TruncatedOracle.sol
- PoolInfo.sol
- TokenOptions.sol
- Common.sol
- CommonBlueprint.sol
- CommonUpgradeable.sol
- OperatorBlueprint.sol
- OperatorUpgradeable.sol

The following number of issues were found, categorized by their severity:

- Critical & High: 12 issues
- Medium: 13 issues
- Low: 15 issues
- Informational: 3 issues

Findings Summary

ID	Title	Severity	Status
[C-01]	Critical precision loss in launch refunds	Critical	Fixed
[C-02]	CTO inactivity guard reads from wrong storage mapping - any token takeable immediately	Critical	Fixed
[C-03]	`processPayment` allows arbitrary sender in ERC20 payments	Critical	Fixed

ID	Title	Severity	Status
[H-01]	`BDeFiLaunch.sol#voidLaunch()` does not decrement `ethParticipation` after sending the void fee - refund pool is insolvent	High	Fixed
[H-02]	Free launch feature is permanently broken due to conflicting validation constraints	High	Fixed
[H-03]	`USD_CTO_FEE` misconfigured - CTO costs $0.08 instead of $8.00	High	Fixed
[H-04]	`enableTokenCto` never zeroes the tokens pool balance - phantom balance drains other tokens leaderboard funds	High	Fixed
[H-05]	`BUY_AND_BURN` fees from external tokens are permanently locked in `BDeFiHook`	High	Fixed
[H-06]	`finalizeEpoch` deploys a `BuyBurn` without registering it - orphaned ETH and permanent epoch lock on second win	High	Fixed
[H-07]	CTO feature can be continuosly DOSed via dust swaps	High	Acknowledged
[H-08]	`BDeFiWhitelist` calls wrong initializer so the paid external whitelist permanently broken	High	Fixed
[H-09]	Uniform slippage applied to variable swap amount during tax processing	High	Fixed
[M-01]	Unscaled addition in `BDeFiAaveStaking._availableSupply` severely underestimates Aave's actual total supply	Medium	Fixed
[M-02]	Rewards get stuck in BdefiAave staking contract	Medium	Fixed
[M-03]	When an external token is removed from whitelist its token fulfilment queues are not cleared	Medium	Acknowledged
[M-04]	Missing collect trading fees before removing external whitelist	Medium	Acknowledged
[M-05]	Sell-side tax bypassed during oracle warmup period	Medium	Fixed
[M-06]	`processExternalToken` strict equality check enables persistent griefing DoS	Medium	Fixed
[M-07]	Yield distribution to internal staking can be sniped for quick profits	Medium	Acknowledged
[M-08]	`updateUserProfile` EIP712 signature verification omits profile metadata strings	Medium	Fixed
[M-09]	No functionality to fully withdraw all externally staked tokens	Medium	Fixed

ID	Title	Severity	Status
[M-10]	Logic for buyburn speed is not implemented	Medium	Acknowledged
[M-11]	Removal of whitelist for internal tokens is possible which is not intended	Medium	Fixed
[M-12]	Removal of external primary token can prevent launch of an internal token	Medium	Fixed
[M-13]	When finalizing epoch it is not checked that each winner is an internal token or not which can lead to wastage of rewards	Medium	Fixed
[L-01]	`claimFor()` ETH fallback silently redirects user funds to leaderboard	Low	Fixed
[L-02]	Missing pending check in external whitelist purchase allows fee wastage	Low	Fixed
[L-03]	Incorrect isParentExternal boolean set for external pools	Low	Fixed
[L-04]	New observation is written even when fees are collected from the pools	Low	Fixed
[L-05]	Missing collecttradingfees call in beforeSwap in hook contract	Low	Acknowledged
[L-06]	Config setters accross the protocol accept arbitrary values and admin misconfiguration will cause permanent protocol DoS	Low	Acknowledged
[L-07]	`setBlueprint` unconditionally increments version so duplicate calls shift all future CREATE2 addresses	Low	Fixed
[L-08]	Removing whitelisted external parent permanently breaks swap routes for all child tokens	Low	Acknowledged
[L-09]	`_liquidityActionV4` uses `block.timestamp` for currency1 permit2 approval instead of `deadline`	Low	Fixed
[L-10]	Epoch 0 can never be finalized - first two weeks of leaderboard earnings permanently locked	Low	Fixed
[L-11]	The `uint64` parameter in `setPointsPack` prevents admin from restoring default pack sizes	Low	Fixed
[L-12]	Stablecoin depeg risk due to hardcoded 1:1 conversion	Low	Acknowledged
[L-13]	LIFO fee queue processing leads to permanent fee starvation	Low	Acknowledged
[L-14]	Creator can receive additional support allocation, exceeding intended creator cap	Low	Acknowledged

ID	Title	Severity	Status
[L-15]	Missing operator reimbursement in `distributeTaxes` may stall tax distribution	Low	Acknowledged
[I-01]	Hardcoded mainnet addresses in `Common.sol` will break multichain deployments	Informational	Acknowledged
[I-02]	`MAX_ETH_PRICE_STALENESS` default value is high for ETH/USD price feed	Informational	Fixed
[I-03]	Typo: `hanlder` instead of `handler`	Informational	Fixed

Detailed Findings

[C-01] Critical precision loss in launch refunds

Severity

Impact: High

Likelihood: High

Description

In `BDeFiLaunch.sol`, the `refundTokens` function calculates the proportion of ETH a user should be refunded when a launch is voided. However, there is a severe precision loss bug in how the user's basis points (BPS) are calculated.

```
    uint256 tokenSupply = bdefiToken.totalSupply();
    uint256 tokenAmount = bdefiToken.balanceOf(msg.sender);
    uint256 userBps = (tokenAmount * 100) / tokenSupply; // <== Percentage
not BPS

    uint256 ethRefund = _applyBps(info.ethParticipation, uint16(userBps));
```

The code multiplies by `100`, which calculates a standard percentage (e.g., 50% = 50). It then passes this directly into `_applyBps()`, which expects a basis point representation where 100% = `10000`. By passing `100` instead of `10000`, the refund amount shrinks by a factor of 100, causing users to lose 99% of their rightful ETH refund. Furthermore, any user holding less than 1% of the supply will have `userBps` truncated to `0`, receiving nothing.

Recommendations

Update the multiplier to properly reflect basis points (`10000` or `BPS_BASE`):

```
uint256 userBps = (tokenAmount * 10000) / tokenSupply;
```

[C-02] CTO inactivity guard reads from wrong storage mapping - any token takeable immediately

Severity

Impact: High

Likelihood: High

Description

`BDeFiConfig._setDefaultSettings()` writes the 30-day inactivity window into `uint256Values`:

```
uint256Values[BDeFiConfigNames.CTO_MIN_INACTIVE_TIME] = 30 days;
```

But `BDeFiLeaderboard.enableTokenCto()` reads it from `uint16Values`:

```
uint256 minInactiveTime =
config.uint16Values(BDeFiConfigNames.CTO_MIN_INACTIVE_TIME); // → 0
```

`uint16Values[CTO_MIN_INACTIVE_TIME]` is never initialized, so it returns `0`. The guard `block.timestamp - lastTradedT < 0` is `uint256` arithmetic and can never be true. The check is permanently bypassed.

Two additional facts make this critical: `enableTokenCto()` has no access control (anyone can call it directly), and `lastTradedTs[token]` is set at pool initialization - not after the first trade, so every token is immediately eligible.

A complete attack from launch:

1. Token launches, pool initializes -> `lastTradedTs[token]` is set
2. Attacker calls `Leaderboard.enableTokenCto(token)` directly - zero wait, no payment
3. The token's entire leaderboard ETH pool is split and partially sent to `protocolGenesis` (irreversibly)
4. Attacker calls `Core.purchaseCtoToken()` paying $0.08 (see H-01) to take ownership

Secondary issue: even if the mapping were fixed, `uint16` max is 65,535 seconds (~18 hours) and cannot represent 30 days (2,592,000 seconds), so the storage type also needs to change.

Recommendations

Change the read in `BDeFiLeaderboard.enableTokenCto()` to use the correct mapping:

```
uint256 minInactiveTime =
config.uint256Values(BDeFiConfigNames.CTO_MIN_INACTIVE_TIME);
```

[C-03] `processPayment` allows arbitrary sender in ERC20 payments

Severity

Impact: High

Likelihood: High

Description

The `processPayment` function allows anyone to pass an arbitrary `sender` address as an argument. Any user who has approved the `BDeFiPayments` contract to spend her USDC (e.g to launch a token or buy a boost), stands the risk of being drained by an attacker especially if they had given max approval. The attacker can call the function and pass themselves as a referrer. The contract will pull the tokens directly from victim. and instantly reward the attacker with part of the stolen funds as a referral fee, while the remaining goes to the protocol genesis.

```
function processPayment(address sender, address paymentToken, uint256
usdAmount, address referrer)
    external payable nonReentrant returns (uint256 tokenAmount)
```

Inside `_processUsdPeggedToken`, the code executes:

```
    token.safeTransferFrom(sender, receiver, paymentAmount);
```

Recommendations

Validate the function caller and limit it to only BDeFi approved contracts, Core, Whitelist, Points, etc

[H-01] `BDeFiLaunch.sol#voidLaunch()` does not decrement `ethParticipation` after sending the void fee - refund pool is insolvent

Severity

Impact: High

Likelihood: Medium

Description

`BDeFiLaunch.sol#voidLaunch()` sends the 8% void fee out of the contract but never subtracts it from `info.ethParticipation`. That stale balance is then used as the base for every refund calculation in `BDeFiLaunch.sol#refundTokens()`, so the sum of all refunds the contract promises is larger than what it actually holds.

The last user(s) to claim will hit an insufficient balance and their `_safeEthTransfer` will revert. Since `BDeFiToken.sol#handleRefund()` burns their tokens before the transfer attempt, the whole transaction reverts - tokens are not lost, but that user simply can never claim. The void fee ETH (0.8 ETH on a 10 ETH launch) is permanently locked with no rescue path in `BDeFiLaunch.sol`.

Recommendations

Decrement `ethParticipation` immediately after sending the void fee in `BDeFiLaunch.sol#voidLaunch()`:

```
if (voidFee > 0) {

_safeEthTransfer(bdefiCore().getContract(BDeFiContractNames.LEADERBOARD),
voidFee);
    info.ethParticipation -= voidFee;
}
```

[H-02] Free launch feature is permanently broken due to conflicting validation constraints

Severity

Impact: High

Likelihood: Medium

Description

`BDeFiConfig.sol#validateLaunchOptions()` runs `BDeFiConfig.sol#_validateEthAllocations()` before `BDeFiConfig.sol#_validateFreeLaunch()`. These two functions impose mutually exclusive requirements on `opts.ethAllocs.addSupport`:

- `BDeFiConfig.sol#_validateEthAllocations()` rejects any `addSupport > BPS_MAX_SUPPORT_ALLOC` (default: `8_00`)
- `BDeFiConfig.sol#_validateFreeLaunch()` requires `addSupport == MAX_ADD_TOKEN_LP_MINT_BPS` (default: `28_00`)

Since `28_00 > 8_00`, the first validator always reverts before the second is ever reached. There is no value of `addSupport` that satisfies both conditions. Every call with `isFreeLaunch = true` reverts unconditionally.

`MAX_ADD_TOKEN_LP_MINT_BPS` is also the wrong variable - it controls extra token minting for LP initialization, not ETH allocation percentages. `freeLaunchEnabled`, `freeLaunchCount` and `MAX_FREE_LAUNCHES_PER_ADDRESS` are all dead storage.

Recommendations

Change `BDeFiConfig.sol#_validateFreeLaunch()` to compare against `BPS_MAX_SUPPORT_ALLOC` (or a dedicated free-launch constant), not `MAX_ADD_TOKEN_LP_MINT_BPS`:

```
if (opts.ethAllocs.addSupport !=
uint16Values[BDeFiConfigNames.BPS_MAX_SUPPORT_ALLOC]) {
    revert BDeFiConfig__InvalidSupportAllocation();
}
```

[H-03] `USD_CTO_FEE` misconfigured - CTO costs $0.08 instead of $8.00

Severity

Impact: High

Likelihood: Medium

Description

Every USD fee constant in `BDeFiConfig._setDefaultSettings()` uses the two-decimal format (e.g., `8_00` = $8.00). `USD_CTO_FEE` is the only exception:

```
uint16Values[BDeFiConfigNames.USD_CTO_FEE] = 8; // comment says "8$" —
actually $0.08
```

With `USD_DECIMALS = 2`, the value `8` means $0.08. At a $3,000 ETH price, the buyer pays 0.000027 ETH instead of 0.002667 ETH - 100× underpaid.

Combined with the "CTO inactivity guard reads from wrong storage mapping" issue, any token can be taken over immediately for $0.08.

Recommendations

Change:

```
uint16Values[BDeFiConfigNames.USD_CTO_FEE] = 8_00; // $8.00
```

[H-04] `enableTokenCto` never zeroes the tokens pool balance - phantom balance drains other tokens leaderboard funds

Severity

Impact: High

Likelihood: Medium

Description

When `BDeFiLeaderboard.sol#enableTokenCto()` is called, it reads the token's accumulated leaderboard pool balance and splits it, crediting half to `tokenPoolBalances[ETH]` and sending the other half directly to `protocolGenesis`. The problem is that `tokenPoolBalances[token]` is never decremented after this split. The ETH is physically gone from the contract, but the accounting still shows the full original balance under that token.

```
uint256 poolBalance = tokenPoolBalances[token];
uint256 ethLeaderboardAmount = poolBalance / 2;
uint256 genesisAmount = poolBalance - ethLeaderboardAmount;

tokenPoolBalances[ETH] += ethLeaderboardAmount;
if (genesisAmount > 0) _safeEthTransfer(config.protocolGenesis(),
genesisAmount);
// tokenPoolBalances[token] is never touched
```

From this point forward the invariant `sum(tokenPoolBalances) == address(leaderboard).balance` is broken. The leaderboard is now accounting for more ETH than it holds by exactly `poolBalance`.

After a CTO purchase, `BDeFiLeaderboard.sol#handleTokenCtoPurchase()` clears the CTO flag and the token continues trading. New fees from `BDeFiLeaderboard.sol#allocateTradingFees()` accumulate on top of the ghost balance. When `BDeFiLeaderboard.sol#finalizeEpoch()` eventually pays out for this token it draws ETH that doesn't exist in its allocation - the shortfall is covered by physically draining ETH that belongs to other tokens' pools, leaving them underpaid or causing their own `finalizeEpoch` calls to revert.

Recommendations

Zero the tokens pool balance immediately after computing the split in `BDeFiLeaderboard.sol#enableTokenCto()`:

```
tokenPoolBalances[token] = 0;
tokenPoolBalances[ETH] += ethLeaderboardAmount;
if (genesisAmount > 0) _safeEthTransfer(config.protocolGenesis(),
genesisAmount);
```

[H-05] BUY_AND_BURN fees from external tokens are permanently locked in BDeFiHook

Severity

Impact: High

Likelihood: Medium

Description

In BDeFiLpManagerActions.sol, when trading fees are distributed and a custom LP fee is allocated to LpFeeReceivers.BUY_AND_BURN, the code attempts to route the parent token share to the Buy&Burn contract.

```
        if (receiver == LpFeeReceivers.BUY_AND_BURN) {
            // burn all tokens
            IERC20Burnable(poolCoreInfo.token).burn(receiverTokenShare);
            // parent tokens allocated for Buy And Burn
            address buyBurn =
bdefiCore().getTokenModule(poolCoreInfo.token,
BDeFiContractNames.BUY_BURN);
            _addToHookFullfilmentQueue(
                poolCoreInfo.parentToken, buyBurn, receiverParentShare,
false, false, hook, leaderboard
            );
        }
```

When poolCoreInfo.parentToken is an external token, it enters the fulfillment queue. Later, when an operator calls processExternalToken, the tokens are swapped to ETH and distributed. Since isLeaderboard was set to false, the Hook contract accounts for the ETH by adding it to the buyBurn contract's claimable balance:

```
        if (item.isLeaderboard)
leaderboard.allocateTradingFees(item.receiver, itemEthAmount);
        else claimableEth[item.receiver] += itemEthAmount;
```

To extract this ETH, the buyBurn contract must explicitly call hook.claimFees(). However, the BDeFiBuyBurn.sol contract does not contain any function to invoke claimFees() on the Hook contract.

Consequently, any fees converted from external parent tokens and routed to `BUY_AND_BURN` will be permanently locked inside the Hook contract.

Recommendations

Implement a function within `BDeFiBuyBurn.sol` that calls `IBDeFiHook(hook).claimFees()` to properly pull and aggregate accumulated ETH prior to executing the buy and burn.

[H-06] `finalizeEpoch` deploys a `BuyBurn` without registering it - orphaned ETH and permanent epoch lock on second win

Severity

Impact: High

Likelihood: Medium

Description

When `BDeFiLeaderboard.sol#finalizeEpoch()` finds a winning token with no `BuyBurn` module, it deploys one via `BDeFiCloner.sol#deployBuyBurn()` and immediately sends the payout ETH to it. The deployed address is never written to `BDeFiCore.sol#_tokenModules`. `BDeFiCore.sol` has no `setTokenModule` function that mapping is only written during `BDeFiCore.sol#launch()`.

Two consequences follow: First win: ETH lands in a contract that has no registered reference anywhere in the protocol. While the address is theoretically derivable from the CREATE2 formula, there is no protocol mechanism to operate it.

Second win: `getTokenModule(winner, BUY_BURN)` still returns `address(0)`. `BDeFiCloner.sol#deployBuyBurn()` is called again with the identical salt `keccak256(winner, blueprintVersion, chainId)` and `Clones.cloneDeterministic` reverts because code already exists at that address. The entire `finalizeEpoch` transaction reverts and the epoch is permanently unresolvable, since the winner list cannot be changed.

Recommendations

Add a `setTokenModule` function to `BDeFiCore.sol` restricted to the Leaderboard and call it after deployment in `BDeFiLeaderboard.sol#finalizeEpoch()`:

```
buyBurn = IBDeFiCloner(...).deployBuyBurn(winner, BuyBurnSpeed.MEDIUM);
bdefiCore().setTokenModule(winner, BDeFiContractNames.BUY_BURN, buyBurn);
```

[H-07] CTO feature can be continuosly DOSed via dust swaps

Severity

Impact: Medium

Likelihood: High

Description

The `enableTokenCto` function in `BDeFiLeaderboard.sol` allows the community to take over an inactive token if it hasn't been traded for a period defined by `CTO_MIN_INACTIVE_TIME`:

```
        uint256 lastTradedT =
IBDeFiHook(bdefiCore().getContract(BDeFiContractNames.HOOK)).lastTradedTs(
token);
        if (lastTradedT == 0) revert
BDeFiLeaderboard__TokenNotInitialized();

        uint256 minInactiveTime =
config.uint16Values(BDeFiConfigNames.CTO_MIN_INACTIVE_TIME);
        if (block.timestamp - lastTradedT < minInactiveTime) revert
BDeFiLeaderboard__CtoUnavailable();
```

The `lastTradedTs` variable is updated in `BDeFiHook.sol` during the `_beforeSwap` hook:

```
    function _beforeSwap(...) internal override returns (...) {
        ...
        lastTradedTs[poolCoreInfo.token] = block.timestamp;
```

A malicious user can continuosly prevent a CTO by simply executing a 1 wei swap (dust swap) on the token's pool once every 29 days. This updates the `lastTradedTs` to `block.timestamp`, repeatedly denying the community the ability to trigger the CTO logic.

Recommendations

Implement a minimum volume threshold over a given time period or a minimum trade size required to update the `lastTradedTs` timestamp.

[H-08] `BDeFiWhitelist` calls wrong initializer so the paid external whitelist permanently broken

Severity

Impact: High

Likelihood: Medium

Description

`BDeFiWhitelist.sol` inherits `OperatorUpgradeable` but `initialize()` calls `__Common_init` instead of `__Operator_init`. The consequence is that `OperatorStorage.contractNameHash` is never set and stays `bytes32(0)`.

`BDeFiWhitelist.sol#operator()` resolves as:

```
return bdefiCore().getOperator(_getOperatorStorage().contractNameHash);
// -> bdefiCore().getOperator(bytes32(0)) → address(0)
```

`BDeFiWhitelist.sol#purchaseExternalWhitelist()` verifies the operator signature with:

```
if (signer != operator()) revert BDeFiWhitelist__InvalidSignature();
// operator() = address(0)
// signer (from any real signature) != address(0) -> ALWAYS REVERTS
```

Every call to `purchaseExternalWhitelist()` permanently reverts regardless of signature validity. The entire paid external whitelist flow - the $288 fee path is dead from deployment. No external token can ever be whitelisted through the purchase mechanism. Admin-direct whitelisting via `BDeFiWhitelist.sol#whitelistExternalToken()` still works since it doesnt verify an operator signature.

Recommendations

Change `BDeFiWhitelist.sol#initialize()` to call the correct initializer with the whitelist contract name:

```
function initialize(address _bdefiCore, uint16 _version) external
initializer {
    __Operator_init(_bdefiCore, _version, BDeFiContractNames.WHITELIST);
    __EIP712_init("BuilDefi", "1");

    tokenWhitelistTypes[ETH] = WhitelistType.INTERNAL;
}
```

[H-09] Uniform slippage applied to variable swap amount during tax processing

Severity

Impact: High

Likelihood: Medium

Description

In `BDeFiTaxDistributor.sol`, the `distributeTaxes` function loops over an array of tax destinations and calls `_processTax`. If a destination requires the tax to be swapped to ETH (e.g., external staking or leaderboard pools), it executes the swap using the `minAmountOutV4` and `minAmountOutV3` arguments passed by the operator.

```
        amount = _sanitizeOperatorAmount(token, amount);
        for (uint256 i = 0; i < numDestinations; i++) {
            _processTax(token, allocations[i], amount, minAmountOutV4,
minAmountOutV3, deadline);
        }
```

Because each destination receives a different allocation (e.g., 2% to Leaderboard, 5% to Staking), the `taxAmount` being swapped is completely different for each loop iteration. Applying the exact same static `minAmountOut` to varying input amounts is fundamentally broken. It guarantees that the transaction will either revert (if `minAmountOut` is scaled for the largest swap) or expose the smaller swaps to devastating MEV sandwich attacks (if scaled for the smallest swap).

Recommendations

Do not swap tokens individually within the loop. Instead, aggregate the total amount of tokens that need to be swapped to ETH, execute a single swap using the provided slippage parameters, and then distribute the resulting ETH proportionally to the respective destinations.

[M-01] Unscaled addition in `BDeFiAaveStaking._availableSupply` severely underestimates Aave's actual total supply

Severity

Impact: Medium

Likelihood: Medium

Description

In `BDeFiAaveStaking.sol`, the internal view `_availableSupply` is used to determine how much capacity remains under the Aave pool's supply cap.

```
    function _availableSupply(IPoolDataProvider dataProvider, uint256
supplyCapRaw) internal view returns (uint256) {
...

        uint256 currentTotalSupply = aToken.scaledTotalSupply() +
```

```
accruedToTreasuryScaled.rayMul(liquidityIndex);
...
```

This computation is flawed. `aToken.scaledTotalSupply()` returns the sum of raw scaled balances. To convert it back to the actual underlying token amount, it must be multiplied by the `liquidityIndex` using `.rayMul()`. Because this multiplication is missing for `aToken.scaledTotalSupply()`, the calculated `currentTotalSupply` is drastically understated (by a factor of ~1e27).

This renders the supply cap check useless. The contract will attempt to deposit into Aave even when the cap is reached, causing Aave to revert the transaction. In cases where this deposit occurs from the `receive()` fallback, the entire contract will effectively reject ETH deposits when the Aave pool is full causing DOS on a larger scale when the contract should receive extra rewards.

Recommendations

Fix the calculation to match Aave's internal `ReserveLogic` by grouping both terms before applying `rayMul`:

```
        uint256 currentTotalSupply = (aToken.scaledTotalSupply() +
accruedToTreasuryScaled).rayMul(liquidityIndex);
```

[M-02] Rewards get stuck in BdefiAave staking contract

Severity

Impact: Medium

Likelihood: Medium

Description

Atoken also earn some external rewards which are collected when claim yield is called. The issue is that when these rewards are collected they are transferred to the bdefi aave staking contract but they remain stuck in the contract as there is no function which can retrieve or utilize these reward tokens. Following is claim yield function which claims rewards to self

```
/// @notice Claims the rewards from the Aave pool
    /// @param amount The amount of WETH to withdraw
    function _claimYield(uint256 amount) internal {
        address[] memory assets = new address[](1);
        assets[0] = address(aToken);
        _getRewardsController().claimAllRewardsToSelf(assets);
        _withdrawAsset(amount);
        IWETH(suppliedAsset).withdraw(amount);
```

```
        emit YieldClaimed(amount);
    }
```

Claim other rewards correctly transfers the rewards to genesis so similar logic should be enforced for these tokens as well.

```
/// @notice Claims other accrued rewards, except the underlying asset
    /// @param assets Reward asset list for Aave rewards claim (must be
non-empty and exclude this `aToken`)
    function claimOtherRewards(address[] memory assets) external {
        address aTokenAddress = address(aToken);
        uint256 numAssets = assets.length;
        if (numAssets == 0) revert BDeFiAaveStaking__Prohibited();
        for (uint256 i = 0; i < numAssets; i++) {
            if (assets[i] == aTokenAddress) revert
BDeFiAaveStaking__Prohibited();
        }

        _getRewardsController().claimAllRewards(assets,
bdefiConfig().protocolGenesis());
    }
```

Recommendations

Implement a function like an admin only or genesis restricted to retrieve these reward tokens.

[M-03] When an external token is removed from whitelist its token fulfilment queues are not cleared

Severity

Impact: Medium

Likelihood: Medium

Description

When an external token is removed its token fulfilment queue can still have some elements in it and once the external token is removed process external token in hooks contract would be useless as swap external token to eth call will fail as swap handler doesn't swap non whitelisted external tokens due to this those tokens allocated will remain stuck in the contract.

Recommendations

Before removing external whitelist process external token or clear the fulfillment queue for that token.

[M-04] Missing collect trading fees before removing external whitelist

Severity

Impact: Medium

Likelihood: Medium

Description

Suppose a whitelisted external token has a parent which is not weth so which means its parent is also external whitelisted so when removing the token from whitelisted if there is pending fees for that pool id it would be lost and as the parent is also externally whitelisted it would mean those parent share tokens would have been utilized for example eth leaderboard or when processing external tokens in hooks contract.Currently there is no call to lp manager for this poolId for collecting trading fees in remove whitelist function.

Recommendations

Add the following line in lp manager handle external whitelist removal function,

```
    /// @notice Clears external pool metadata after whitelist removal.
    /// @param token External token.
    /// @param parentToken Parent token paired with token.
    function handleExternalWhitelistRemoval(address token, address
parentToken) external onlyWhitelist {
        PoolId poolId = _tokenPoolKeys[token][parentToken].toId();
        ExternalPoolType poolType = _externalPoolTypes[poolId];
   +++  collectPoolTradingFees(poolId);

        if (poolType == ExternalPoolType.NONE) revert
BDeFiLpManager__InvalidPool();
        _poolCoreInfos[poolId].poolType = PoolType.NONE;
        IERC20(token).forceApprove(address(PERMIT2), 0);

        emit ExternalPoolRemoved(token, poolId, poolType);
    }
```

[M-05] Sell-side tax bypassed during oracle warmup period

Severity

Impact: Medium

Likelihood: Medium

Description

`BDeFiHook.sol#_beforeSwap()` returns early with `ZERO_DELTA` when `HookLibrary.sol#isObservationMissing()` is true, which it is for the entire first 15 minutes after pool initialization (`DEFAULT_LOOKBACK_SECONDS`). The comment says "skip fulfillment," but the early return sits above the tax logic, so it skips both.

The sell-side tax has no other collection path. `BDeFiHook.sol#_afterSwap()` always runs but resolves the tax currency to ETH for a sell (the output token) and ETH has `tokenTaxBps = 0`, so nothing is collected there either. Buy-side tax is unaffected because `_afterSwap` resolves to the BDeFi token for a buy, which has `tokenTaxBps > 0`.

The first 15 minutes of trading is typically the highest-volume period of any launch. Every sell during that window is tax-free regardless of what the token creator configured.

Recommendations

Move `_applyBuySellTax` above the `isObservationMissing` check in `BDeFiHook.sol#_beforeSwap()`. Tax computation doesn't use the oracle at all. It only needs `params.amountSpecified` and `tokenTaxBps`, so there's no reason it should be gated behind it:

```
specifiedDelta = _applyBuySellTax(params, poolKey, BalanceDelta.wrap(0),
true);

if (HookLibrary.isObservationMissing(poolKey, lookbackTime)) {
    return (this.beforeSwap.selector,
toBeforeSwapDelta(int128(specifiedDelta), 0), 0);
}

// fulfillment logic below (requires TWAP)
```

[M-06] `processExternalToken` strict equality check enables persistent griefing DoS

Severity

Impact: Medium

Likelihood: Medium

Description

`BDeFiHook.sol#processExternalToken()` requires that the total tokens distributed across genesis fees and the fulfillment queue equals exactly the operator-supplied `fulfilmentAmount`:

```
if (totalAmountFulfilled != fulfilmentAmount) revert
BDeFiHook__InvalidFulfillmentAmount();
```

The operator computes `fulfilmentAmount` off-chain by reading `genesisFulfillmentAmounts[token]` and the current queue state. Between that read and on-chain execution, any swap involving the token updates `genesisFulfillmentAmounts[token]`. The on-chain distribution then produces a different total, causing the transaction to revert.

An adversary who wants to prevent external fee distribution for a specific token can continuously trigger small swaps to keep `genesisFulfillmentAmounts[token]` in constant flux. Every operator submission lands with a stale `fulfilmentAmount` and reverts. The cost to the attacker is only the swap fees on small amounts. The cost to the protocol is that accumulated fees are never distributed and the receivers are permanently cut off from their allocations for as long as the griefing continues.

Recommendations

Replace the strict equality with a `>=` check to allow partial fulfillment and eliminate the griefing surface:

```
if (totalAmountFulfilled > fulfilmentAmount) revert
BDeFiHook__InvalidFulfillmentAmount();
```

Alternatively, remove the caller-supplied `fulfilmentAmount` parameter entirely and have `BDeFiHook.sol#processExternalToken()` derive the correct distribution amount from live on-chain state at execution time.

[M-07] Yield distribution to internal staking can be sniped for quick profits

Severity

Impact: Medium

Likelihood: Medium

Description

In `BDeFiAaveStaking.sol`, operators distribute accrued yield by swapping it into Internal BDeFi Tokens and sending it to the Internal Staking module:

```
        } else if (_yieldReceiver ==
ExternalStakeYieldReceiver.INTERNAL_STAKING) {
            address internalStaking =
bdefiCore().getTokenModule(bdefiToken, BDeFiContractNames.INT_STAKING);

IBDeFiSwapHandler(bdefiCore().getContract(BDeFiContractNames.SWAP_HANDLER)
```

```
)
            .swapEthToInternalToken{value: amount}(
                bdefiToken, minAmountOutV4, minAmountOutV3,
internalStaking, deadline
            );
```

Additionally, in `BDeFiTaxDistributor.sol`, if the tax receiver is `INTERNAL_STAKING`, the tokens are sent to the internal staking module.

```
        } else if (receiver == TokenTaxReceivers.INTERNAL_STAKING) {
            beneficiary = bdefiCore().getTokenModule(tokenAddress,
BDeFiContractNames.INT_STAKING);
        } else if (receiver == TokenTaxReceivers.TOKEN_CREATOR) {
            beneficiary = bdefiCore().getTokenOwner(tokenAddress);
        }

        if (beneficiary != ZERO_ADDRESS) {
            token.safeTransfer(beneficiary, taxAmount);
            return;
        }
```

The `BDeFiStaking` module is implemented as an `ERC4626Upgradeable` vault. When tokens are sent directly to the contract via the vault's assets increase without minting new shares acting as yield for stakers.

Opportunistic users can monitor for these operations, frontrun the transaction by depositing into the `BDeFiStaking` contract (acquiring a large portion of the vault's shares), let the yield land, and then immediately withdraw in the same or next block. This allows attackers to steal a large percentage of the distributed yield that was meant for long-term stakers without meaning fully contributing to the protocol.

Recommendations

The `BDeFiStaking` contract might need to be modified, e.g making it a time based staking system that calculates rewards based on how long a user has staked for.

[M-08] `updateUserProfile` EIP712 signature verification omits profile metadata strings

Severity

Impact: Low

Likelihood: High

Description

In `BDeFiPoints.sol`, `updateUserProfile` uses an EIP712 signature to authorize profile modifications. However, the encoded digest completely omits the actual metadata being updated (`username`, `avatar`, `description`).

```
bytes32 digest =
    _hashTypedDataV4(keccak256(abi.encode(PROFILE_UPDATE_TYPEHASH,
msg.sender, feeRequired, nonce)));
```

A malicious user can request authorization for innocuous profile details off-chain, receive the operator's signature, and then broadcast the transaction on-chain swapping the arguments for more malicious strings (e.g offensive content). The signature will still validate because the string inputs are not part of the hashed digest.

Recommendations

Include the `username`, `avatar`, and `description` in the hash validation.

[M-09] No functionality to fully withdraw all externally staked tokens

Severity

Impact: High

Likelihood: Low

Description

`BDeFiAaveStaking.deposit()` allows sending and staking all tokens to aave pool. In a case where it is needed, there is no functionality to withdraw all tokens from aave. No other function can withdraw unstaked WETH. We can only claim yield, and distribute it. At worst, `updateAavePool` only moves assets from one pool to the next. And if deposit to the new pool is unavailable, withdrawn WETH is stuck in the contract.

Recommendations

Introduce an admin controlled function to access the `_withdrawAsset` function.

[M-10] Logic for buyburn speed is not implemented

Severity

Impact: Medium

Likelihood: Medium

Description

There is key feature in buy burn contract called buy burn speed which is the speed at which the buy burn contract should burn its balance but when burning its token it is no where checked that the buy burn happened at the correct speed which makes the buy burn speed variable redundant. It is clear that the speed logic was not implemented.

```
*//////////////////////////////////////////////////////////////
                            OPERATOR FUNCTIONS
    //////////////////////////////////////////////////////////////*/
    /// @notice Swaps ETH to BuilDeFi token and burns received amount.
    /// @param amount Total ETH amount to use (type(uint256).max uses full
ETH balance)
    /// @param reimbursement ETH reimbursement for operator execution
(deducted from amount)
    /// @param minAmountOutV4 Minimum output for V4 route
    /// @param minAmountOutV3 Minimum output for V3 route
    /// @param deadline Swap deadline timestamp
    function buyBurn(
        uint256 amount,
        uint256 reimbursement,
        uint256 minAmountOutV4,
        uint256 minAmountOutV3,
        uint256 deadline
    ) external onlyOperator {
        amount = _sanitizeOperatorAmount(amount);
        _checkReimbursement(amount, reimbursement);

        uint256 amountOut =
IBDeFiSwapHandler(bdefiCore().getContract(BDeFiContractNames.SWAP_HANDLER)
)
        .swapEthToInternalToken{value: amount - reimbursement}(
            address(bdefiToken), minAmountOutV4, minAmountOutV3,
address(this), deadline
        );

        burn();
        _safeEthTransfer(operator(), reimbursement);

        emit BuyBurn(address(bdefiToken), amount, amountOut);
    }
```

It is clear from above buy burn function that no where the speed variable is taken into consideration while calling this function.

Recommendations

Implement the logic for buy burn speed.

[M-11] Removal of whitelist for internal tokens is possible which is not intended

Severity

Impact: Medium

Likelihood: Medium

Description

Confirmed with the devs that the removal of internal tokens shouldn't be possible even though the remove whitelist token is only admin restricted , still it should not be possible for admin to perform this action.

Recommendations

Add a check which prevents removal of whitelist for internal tokens,

[M-12] Removal of external primary token can prevent launch of an internal token

Severity

Impact: Medium

Likelihood: Medium

Description

When an internal token is launched it can have its primary token as one of the externally whitelisted token but admin can remove externally whitelisted tokens as can be seen below

```
    /// @notice Removes token from whitelist and clears external
metadata/routes when applicable.
    /// @param token Token to remove
    function removeWhitelistToken(address token) external onlyAdmin {
        WhitelistType tokenType = tokenWhitelistTypes[token];

        if (tokenType == WhitelistType.DISABLED) revert
BDeFiWhitelist__InvalidToken();

        if (tokenType == WhitelistType.EXTERNAL) {
            ExternalPoolInfo storage info = _externalPoolInfos[token];
            address parent = info.parent;


IBDeFiLpManager(bdefiCore().getContract(BDeFiContractNames.LP_MANAGER))
```

```
                .handleExternalWhitelistRemoval(token, parent);

            delete _externalPoolInfos[token];
            delete _externalBuyRoutes[token];
            delete _externalSellRoutes[token];
        }

        delete tokenWhitelistTypes[token];
        emit TokenStatusUpdated(token, WhitelistType.DISABLED);
    }

/// @notice Clears external pool metadata after whitelist removal.
    /// @param token External token.
    /// @param parentToken Parent token paired with token.
    function handleExternalWhitelistRemoval(address token, address
parentToken) external onlyWhitelist {
        PoolId poolId = _tokenPoolKeys[token][parentToken].toId();
        ExternalPoolType poolType = _externalPoolTypes[poolId];

        if (poolType == ExternalPoolType.NONE) revert
BDeFiLpManager__InvalidPool();
        _poolCoreInfos[poolId].poolType = PoolType.NONE;
        IERC20(token).forceApprove(address(PERMIT2), 0);

        emit ExternalPoolRemoved(token, poolId, poolType);
    }
```

It also set the pooltype to none in lp manager contract as well. Now issue is suppose that the internal token has reached minimum participation and also the launch end time has passed but its primary external token has been removed from whitelist, now distribute function is called when closing the launch

```
/// @notice Closes a fair launch after end time and distributes collected
ETH.
    /// @param token BuilDeFi token address whose launch is being closed
    function closeFairLaunch(address token) external nonReentrant {
        LaunchInfo storage info = _launchInfos[token];
        if (info.status != LaunchStatus.OPEN) revert
BDeFiLaunch__LaunchInactive();
        if (block.timestamp <= info.endsAt) revert
BDeFiLaunch__LaunchActive();

        info.status = LaunchStatus.CLOSED;
        _distribute(token, info.ethParticipation);

        emit LaunchStatusUpdate(token, info.status);
    }
 ////////////////////////////////////////////////////////////*/
    /// @notice Distributes collected launch ETH according to configured
allocations.
    /// @param token BuilDeFi token address
```

```
    /// @param ethAmount Total ETH amount to distribute
    function _distribute(address token, uint256 ethAmount) internal {
        EthMintAllocations storage allocs = _mintAllocations[token];
        uint256 totalDistributed;

        if (allocs.creator > 0) {
            uint256 creatorAlloc = _applyBps(ethAmount, allocs.creator);
            totalDistributed += creatorAlloc;

IBDeFiPayouts(bdefiCore().getContract(BDeFiContractNames.PAYOUTS)).registe
rPayout{value: creatorAlloc}(
                bdefiCore().getTokenOwner(token), ETH, creatorAlloc
            );
        }

        if (allocs.addSupport > 0) {
            uint256 supportTotalAlloc = _applyBps(ethAmount,
allocs.addSupport);
            uint256[] memory supportAllocs =
bdefiCore().getTokenAdditionalSupport(token);
            uint256 numAllocs = supportAllocs.length;
            IBDeFiPayouts payouts =
IBDeFiPayouts(bdefiCore().getContract(BDeFiContractNames.PAYOUTS));

            for (uint256 i = 0; i < numAllocs; i++) {
                (address receiver, uint16 bps) =
supportAllocs[i].toAddressAlloc();
                uint256 supportAmount = _applyBps(supportTotalAlloc, bps);
                totalDistributed += supportAmount;
                payouts.registerPayout{value: supportAmount}(receiver,
ETH, supportAmount);
            }
        }

        if (allocs.buyBurn > 0) {
            uint256 buyBurnAlloc = _applyBps(ethAmount, allocs.buyBurn);
            totalDistributed += buyBurnAlloc;
            _safeEthTransfer(bdefiCore().getTokenModule(token,
BDeFiContractNames.BUY_BURN), buyBurnAlloc);
        }

        if (allocs.externalStake > 0) {
            uint256 externalStakeAlloc = _applyBps(ethAmount,
allocs.externalStake);
            totalDistributed += externalStakeAlloc;
            _safeEthTransfer(bdefiCore().getTokenModule(token,
BDeFiContractNames.EXT_STAKING), externalStakeAlloc);
        }

        uint256 platformLbAlloc = _applyBps(ethAmount, allocs.platformLb);

_safeEthTransfer(bdefiCore().getContract(BDeFiContractNames.LEADERBOARD),
platformLbAlloc);
        totalDistributed += platformLbAlloc;
```

```
        IBDeFiLpManager lpManager =
IBDeFiLpManager(bdefiCore().getContract(BDeFiContractNames.LP_MANAGER));

  ===>      if (allocs.primaryTokenSupport > 0) {
            uint256 primaryTokenSupportAlloc = _applyBps(ethAmount,
allocs.primaryTokenSupport);
            totalDistributed += primaryTokenSupportAlloc;
            _handlePrimaryLpDistribution(token, primaryTokenSupportAlloc,
lpManager);
        }

        uint256 lpAllocation = ethAmount - totalDistributed;
        _handleTokenLpsDeposit(token, lpAllocation, lpManager);
    }
```

Now note that the primary token support will always be greater than 0 is the primary token is not eth as can be seen in config contract

```
function _validateEthAllocations(
        TokenLaunchOptions calldata opts,
        TokenDeployModules memory modules,
        address primaryToken
    ) internal view {
        EthMintAllocations calldata allocations = opts.ethAllocs;
        if (allocations.lps <
uint16Values[BDeFiConfigNames.BPS_MIN_LP_ALLOC]) {
            revert BDeFiConfig__InvalidLpEthAllocation();
        }

        if (allocations.creator >
uint16Values[BDeFiConfigNames.BPS_MAX_CREATOR_ALLOC]) {
            revert BDeFiConfig__InvalidCreatorAllocation();
        }

        if (allocations.addSupport > 0) {
            if (allocations.addSupport >
uint16Values[BDeFiConfigNames.BPS_MAX_SUPPORT_ALLOC]) {
                revert BDeFiConfig__InvalidSupportAllocation();
            }
            modules.additionalSupport = true;
        }

        if (allocations.buyBurn > 0) {
            if (allocations.buyBurn >
uint16Values[BDeFiConfigNames.BPS_MAX_BUY_BURN_ALLOC]) {
                revert BDeFiConfig__InvalidBuyBurnAllocation();
            }
            modules.buyBurn = true;
        }
```

```
        if (allocations.externalStake > 0) {
            if (allocations.externalStake >
uint16Values[BDeFiConfigNames.BPS_MAX_EXT_STAKE_ALLOC]) {
                revert BDeFiConfig__InvalidExtStakeAllocation();
            }
            modules.externalStaking = true;
        }

        if (allocations.platformLb !=
uint16Values[BDeFiConfigNames.BPS_PLATFORM_LB_ALLOC]) {
            revert BDeFiConfig__InvalidLbEthAllocation();
        }

        if (
  ===>          allocations.primaryTokenSupport
                != (primaryToken == ETH ? 0 :
uint16Values[BDeFiConfigNames.BPS_PRIMARY_TOKEN_LP_ALLOC])
        ) revert BDeFiConfig__InvalidLbEthAllocation();

        if (
            allocations.lps + allocations.creator + allocations.addSupport
+ allocations.buyBurn
                    + allocations.externalStake + allocations.platformLb +
allocations.primaryTokenSupport != BPS_BASE
        ) revert BDeFiConfig__InvalidEthAllocationSum();
    }
```

So now lets look how primary token support is handled in _distribute function

```
 /// @notice Sends ETH allocation to the primary token LP route.
    /// @param token BuilDeFi token address
    /// @param ethAmount ETH amount to route
    /// @param lpManager LP manager contract instance
    function _handlePrimaryLpDistribution(address token, uint256
ethAmount, IBDeFiLpManager lpManager) internal {
        address primaryToken = bdefiCore().primaryTokens(token);
        address parentToken;

        if (bdefiCore().isExternalToken(primaryToken)) {
            ExternalPoolInfo memory extPoolInfo =
IBDeFiWhitelist(bdefiCore().getContract(BDeFiContractNames.WHITELIST))
                .getExternalPoolInfo(primaryToken);
            lpManager.depositEthToPool{value: ethAmount}(primaryToken,
extPoolInfo.parent);
        } else {
            parentToken = bdefiCore().primaryTokens(primaryToken);
            lpManager.depositEthToPool{value: ethAmount}(primaryToken,
parentToken);
        }
    }
```

If statement will be executed and depositEthTo pool function will be called with primary token and its parent which is as follows

```
/// @notice Deposits ETH directly into a pool runtime balance.
    /// @param token Token mapped to the target pool.
    /// @param parentToken Parent token paired with token in the pool.
    function depositEthToPool(address token, address parentToken) external
payable {
        PoolId poolId = _tokenPoolKeys[token][parentToken].toId();
        if (_poolCoreInfos[poolId].poolType == PoolType.NONE) revert
BDeFiLpManager__InvalidPool();

        _poolBalances[poolId].eth += msg.value;

        emit PoolFunded(token, poolId, _poolCoreInfos[poolId].poolType,
msg.value, 0, 0);
    }
```

Now as the primary token was removed from whitelist pooltype will be none and hence this call will revert which causes closing of launch impossible. As minimum participation has been reached the launch cannot be voided hence the tokens remain permanently stuck in the contract. Due to this i have marked this as high.

Recommendations

Introduce some logic for this specific case which allows the launch to be voided.

[M-13] When finalizing epoch it is not checked that each winner is an internal token or not which can lead to wastage of rewards

Severity

Impact: Medium

Likelihood: Medium

Description

Following is finalizeEpoch function

```
 function finalizeEpoch(
        address token,
        uint256 reimbursement,
        address[] calldata winners,
```

```
        uint16[] calldata winnerBpss
    ) external autoPullEth onlyOperator {
        uint256 currentEpoch = getCurrentEpoch();
        if (finalizedEpochs[token] == currentEpoch) revert
BDeFiLeaderboard__EpochIncomplete();

        uint256 startTs =
            getEpochStart(currentEpoch) +
bdefiConfig().uint256Values(BDeFiConfigNames.EPOCH_FINALIZE_COOLDOWN);
        if (startTs > block.timestamp) revert
BDeFiLeaderboard__EpochCooldown();

        uint256 minPayout =
bdefiConfig().uint256Values(BDeFiConfigNames.LEADERBOARD_PAYOUT_THRESHOLD)
;
        uint16 payoutBps =
bdefiConfig().uint16Values(BDeFiConfigNames.LEADERBOARD_PAYOUT_BPS);
        uint256 totalPayoutAmount = _applyBps(tokenPoolBalances[token],
payoutBps);
        if (totalPayoutAmount < minPayout) revert
BDeFiLeaderboard__InsufficientFunds();

        finalizedEpochs[token] = currentEpoch;
        tokenPoolBalances[token] -= totalPayoutAmount;

        _checkReimbursement(totalPayoutAmount, reimbursement);
        _safeEthTransfer(msg.sender, reimbursement);
        totalPayoutAmount -= reimbursement;

        uint256 totalBps;
        uint256 totalPaidOut;
        uint256 numWinners = winners.length;
        if (numWinners != winnerBpss.length) revert
BDeFiLeaderboard__InvalidAllocations();

        for (uint256 i = 0; i < numWinners; i++) {
            address winner = winners[i];
            uint16 winnerBps = winnerBpss[i];
            uint256 winnerPayout = _applyBps(totalPayoutAmount,
winnerBps);
            address buyBurn = bdefiCore().getTokenModule(winner,
BDeFiContractNames.BUY_BURN);

            if (buyBurn == ZERO_ADDRESS) {
                // deploy a new Buy&Burn if needed
                buyBurn =
IBDeFiCloner(bdefiCore().getContract(BDeFiContractNames.CLONER))
                    .deployBuyBurn(winner, BuyBurnSpeed.MEDIUM);
            }

            totalBps += winnerBps;
            totalPaidOut += winnerPayout;
            _safeEthTransfer(buyBurn, winnerPayout);
        }
```

```
        if (totalBps != BPS_BASE) revert
BDeFiLeaderboard__InvalidAllocations();

        // handle rounding
        if (totalPaidOut < totalPayoutAmount) {
            tokenPoolBalances[token] += totalPayoutAmount - totalPaidOut;
        }

        emit LeaderboardWin(token, winners[0]);
    }
```

Now as can be seen if the winner doesn't have a buy burn it deploys a new buy burn with medium speed but nowhere it is checked that the winner is an internal token. So if one of the winner or all the winners are some external token then a buy burn contract for an external token will be deployed which is of no use and is just a waste of rewards. Now as operators are bots and is no where provided that they are to fully trusted that is why i am reporting this is an issue.

Recommendations

Before deploying buy burn check if the winner is an external token if so revert the transaction.

[L-01] `claimFor()` ETH fallback silently redirects user funds to leaderboard

Description

`BDeFiPayouts.sol#claimFor()` is permissionless so anyone can call it for any account. When the ETH transfer to `account` fails (e.g., the account is a contract without `receive()`), `_safeEthTransferFallback` silently deposits the full amount to the Leaderboard pool instead. The user's `userBalances[account][ETH]` is zeroed before the transfer is attempted. The funds are permanently unrecoverable.

Recommendations

Restrict `claimFor` to only allow the account to claim for itself or skip the fallback-to-leaderboard behavior when called by a third party.

[L-02] Missing pending check in external whitelist purchase allows fee wastage

Description

`BDeFiWhitelist.purchaseExternalWhitelist` allows a user to request whitelisting for an external token by paying a fee. It checks that the token is not already whitelisted, but does not check whether a

pending request already exists for that token. If a user submits the same request twice, or if a malicious user front-runs, the first request is overwritten and the second fee is effectively wasted.

```
    function purchaseExternalWhitelist(
        address token,
        ExternalWhitelistRequest calldata req,
        address paymentToken,
        address referrer,
        bytes calldata signature
    ) external payable nonReentrant {
        if (isWhitelisted(token)) revert
BDeFiWhitelist__TokenIsWhitelisted();
        // ... signature and pool validation ...
        _handlePayment(paymentToken, usdPrice, referrer);
        _pendingExternal[token] = req;
        emit TokenPending(token, msg.sender);
    }
```

Recommendations

- Before assigning `_pendingExternal[token] = req`, check that the current pending request is empty.

[L-03] Incorrect isParentExternal boolean set for external pools

Description

Following is handleExternalWhitelist function

```
/// @notice Registers a newly approved external whitelist pool in LP
manager state.
    /// @param token External token.
    /// @param parentToken Parent token paired with token.
    /// @param poolKey Pool key representing the whitelist-approved pool.
    /// @param isUniswapV4 True when poolKey points to a V4 pool,
otherwise V3 semantics are used.
    function handleExternalWhitelistAdd(address token, address
parentToken, PoolKey calldata poolKey, bool isUniswapV4)
        external
        onlyWhitelist
    {
        PoolId poolId = poolKey.toId();
        ExternalPoolType poolType = isUniswapV4 ? ExternalPoolType.V4 :
ExternalPoolType.V3;

        _tokenPoolKeys[token][parentToken] = poolKey;
        _externalPoolTypes[poolId] = isUniswapV4 ? ExternalPoolType.V4 :
```

```
ExternalPoolType.V3;
        _poolCoreInfos[poolId] = PoolCoreInfo({
            token: token,
            parentToken: parentToken,
            poolType: PoolType.EXTERNAL,
            isParentToken0: uint160(parentToken) < uint160(token),
            isParentExternal: false
        });

        IERC20(token).forceApprove(address(PERMIT2), type(uint256).max);
        emit ExternalPoolAdded(token, poolId, poolType);
    }
```

It is enforced that the parent of externa token can only a external token with pooltype V4 or V3 but while setting the pool core info it incorrectly sets isParentExternal to false whereas it should be true always.

Recommendations

Set isParentExternal to true.

[L-04] New observation is written even when fees are collected from the pools

Description

It is clearly written in the comments of _updatePool function that it should only be called when there is change in pool price or the liqudity as can be seen below

```
/// @dev Called before any action that potentially modifies pool price or
liquidity, such as swap or modify position
    function _updatePool(PoolKey calldata key) private {
        PoolId id = key.toId();
        (, int24 tick,,) = manager.getSlot0(id);

        uint128 liquidity = manager.getLiquidity(id);

        (states[id].index, states[id].cardinality) =
observations[id].write(
            states[id].index, _blockTimestamp(), tick, liquidity,
states[id].cardinality, states[id].cardinalityNext
        );
    }
```

Now when only fees are collected it doesn't modify the pool price of liquidity but it still calls before modify hook which calls _updatePool function. Following is from UniV4 hooks library

```
/// @notice calls beforeModifyLiquidity hook if permissioned and validates
return value
    function beforeModifyLiquidity(
        IHooks self,
        PoolKey memory key,
        ModifyLiquidityParams memory params,
        bytes calldata hookData
    ) internal noSelfCall(self) {
        if (params.liquidityDelta > 0 &&
self.hasPermission(BEFORE_ADD_LIQUIDITY_FLAG)) {
            self.callHook(abi.encodeCall(IHooks.beforeAddLiquidity,
(msg.sender, key, params, hookData)));
        } else if (params.liquidityDelta <= 0 &&
self.hasPermission(BEFORE_REMOVE_LIQUIDITY_FLAG)) {
            self.callHook(abi.encodeCall(IHooks.beforeRemoveLiquidity,
(msg.sender, key, params, hookData)));
        }
    }
```

It can be clearly seen that when liquidity delta is 0 then before remove liquidity function is called in hook.

```
 /// @inheritdoc BaseHook
    function _beforeRemoveLiquidity(address, PoolKey calldata key,
ModifyLiquidityParams calldata, bytes calldata)
        internal
        override
        returns (bytes4)
    {
        // TruncatedGeoOracle tracking
        _beforeModifyPosition(key);
        return this.beforeRemoveLiquidity.selector;
    }
```

and _beforeModifyPosition function calls the update pool function

```
    function _beforeModifyPosition(PoolKey calldata key) internal {
        _updatePool(key);
    }
```

Due to this unnecessary observation is written which only increase the observation array which can even incur way more gas when there is time look up for used for calculation of twap price or for checking missing observation.

Recommendations

In _beforeRemoveLiquidity function check that the liquidity delta is strictly negative.

[L-05] Missing collecttradingfees call in beforeSwap in hook contract

Description

In before swap call if a user is buying a token the contract first tries to fulfill the token request from its internal reserves i.e from genesisFulfillmentAmounts or _tokenFulfillmentQueue[token]. Now issue is that both of these values increase when trading fees is calculated for pools in which token is one of the currency. So when a before swap hook is called the collect trading fees function il bdefi lp manager contract should also be called before calculating how much tokens can be fulfilled from the internal reserves. Following is before swap function

```
function _beforeSwap(address, PoolKey calldata poolKey, SwapParams
calldata params, bytes calldata)
        internal
        override
        returns (bytes4, BeforeSwapDelta, uint24)
    {
        // TruncatedGeoOracle tracking
        _beforeSwap(poolKey);
        PoolId poolId = poolKey.toId();

        PoolCoreInfo memory poolCoreInfo =

IBDeFiLpManager(bdefiCore.getContract(BDeFiContractNames.LP_MANAGER)).getP
oolCoreInfo(poolId);

        if (!fullyInitialized[poolCoreInfo.token]) revert
BDeFiHook__TokenNotFullyInitialized();
        uint256 lookbackTime =
_bdefiConfig().getPoolLookbackConfig(poolId);
        lastTradedTs[poolCoreInfo.token] = block.timestamp;

        // if pool observations are missing, skip fulfillment
        if (HookLibrary.isObservationMissing(poolKey, lookbackTime)) {
            return (this.beforeSwap.selector,
BeforeSwapDeltaLibrary.ZERO_DELTA, 0);
        }

        // Apply buy/sell tax if needed
        int256 specifiedDelta;
        int256 unspecifiedDelta;

        specifiedDelta = _applyBuySellTax(params, poolKey,
BalanceDelta.wrap(0), true);

        // If user is buying the token, try to fulfill from collected fees
first
        if (poolCoreInfo.isParentToken0 == params.zeroForOne) {
            if (params.amountSpecified < 0) {
```

```
                uint160 twapSqrtPriceX96 =
HookLibrary.getTwapSqrtPriceX96(poolKey, uint32(lookbackTime));
                (specifiedDelta, unspecifiedDelta) =
_processInternalTokenFulfillment(
                    specifiedDelta, unspecifiedDelta,
params.amountSpecified, twapSqrtPriceX96, poolCoreInfo
                );
            }
        }

        return (this.beforeSwap.selector,
toBeforeSwapDelta(int128(specifiedDelta), int128(unspecifiedDelta)), 0);
    }
```

it is clear when buying token internal token fulfillment is used before which utilizes genesisFulfillmentAmounts[token] and _tokenFulfillmentQueue[token].

Now lets see the collect trading fees function in lp manager function

```
 /// @param poolId Pool identifier.
    function collectPoolTradingFees(PoolId poolId) external
onlyDelegateCall nonReentrant {
        PoolCoreInfo storage poolCoreInfo = _poolCoreInfos[poolId];
        PoolKey storage poolKey = _tokenPoolKeys[poolCoreInfo.token]
[poolCoreInfo.parentToken];
        PoolPositionInfo storage positionInfo =
_poolPositionInfos[poolId];

        if (poolCoreInfo.poolType == PoolType.NONE) revert
BDeFiLpManager__InvalidPool();

        uint256 tokenFees;
        uint256 parentTokenFees;
        bool isUniswapV3;

        if (poolCoreInfo.poolType == PoolType.EXTERNAL) {
            isUniswapV3 = _externalPoolTypes[poolId] ==
ExternalPoolType.V3;
        }

        (tokenFees, parentTokenFees) = _collectPoolTradingFees(poolKey,
poolCoreInfo, positionInfo, isUniswapV3);
        _distributePoolTradingFees(poolId, tokenFees, parentTokenFees,
poolCoreInfo);
        emit PoolTradingFeesClaimed(poolCoreInfo.token, poolId, tokenFees,
parentTokenFees);
    }
```

It is clear that it can be called by anyone. In distribute pool trading fees function we can clearly see that it increases the _tokenFulfillmentQueue[token] queue

```
function _distributePoolTradingFees(
        PoolId poolId,
        uint256 tokenAmount,
        uint256 parentAmount,
        PoolCoreInfo storage poolCoreInfo
    ) internal {
        IBDeFiHook hook =
IBDeFiHook(bdefiCore().getContract(BDeFiContractNames.HOOK));
        IBDeFiLeaderboard leaderboard =
IBDeFiLeaderboard(bdefiCore().getContract(BDeFiContractNames.LEADERBOARD))
;
        uint256 tokenDistributed;
        uint256 parentDistributed;

        uint256 genesisParentShare = _applyBps(parentAmount,
GENESIS_FEES_BPS);
        _addToHookFullfilmentQueue(
            poolCoreInfo.parentToken, address(0), genesisParentShare,
false, true, hook, leaderboard
        );
        parentDistributed += genesisParentShare;

        uint256[] memory customLpFees =
_customTokenLpFees[poolCoreInfo.token];
        uint256 numCustomFees = customLpFees.length;

        if (numCustomFees > 0) {
            for (uint256 i = 0; i < numCustomFees; i++) {
                (LpFeeReceivers receiver, uint16 bps) =
customLpFees[i].toLpFeeAlloc();
                uint256 receiverTokenShare = _applyBps(tokenAmount, bps);
                uint256 receiverParentShare = _applyBps(parentAmount,
bps);

                if (receiver == LpFeeReceivers.ADD_TO_LP) {
                    PoolBalance storage poolBalance =
_poolBalances[poolId];
                    poolBalance.token += receiverTokenShare;
                    poolBalance.parentToken += receiverParentShare;
                }

                if (receiver == LpFeeReceivers.TOKEN_CREATOR) {
                    address tokenCreator =
bdefiCore().getTokenOwner(poolCoreInfo.token);
                    _addToHookFullfilmentQueue(
                        poolCoreInfo.token, tokenCreator,
receiverTokenShare, false, false, hook, leaderboard
                    );
                    _addToHookFullfilmentQueue(
                        poolCoreInfo.parentToken, tokenCreator,
receiverParentShare, false, false, hook, leaderboard
                    );
```

```
                }

                if (receiver == LpFeeReceivers.BUY_AND_BURN) {
                    // burn all tokens

IERC20Burnable(poolCoreInfo.token).burn(receiverTokenShare);
                    // parent tokens allocated for Buy And Burn
                    address buyBurn =
bdefiCore().getTokenModule(poolCoreInfo.token,
BDeFiContractNames.BUY_BURN);
                    _addToHookFullfilmentQueue(
                        poolCoreInfo.parentToken, buyBurn,
receiverParentShare, false, false, hook, leaderboard
                    );
                }

                tokenDistributed += receiverTokenShare;
                parentDistributed += receiverParentShare;
            }
        }

        // 30% of undistributed parent tokens
        uint256 ethLeaderboardParentShare = _applyBps(parentAmount -
parentDistributed, ETH_LEADERBOARD_FEES_BPS);
        _addToHookFullfilmentQueue(
            poolCoreInfo.parentToken, ETH, ethLeaderboardParentShare,
true, false, hook, leaderboard
        );
        parentDistributed += ethLeaderboardParentShare;

        // rest of parent tokens go to primary token leadeboard
        address primaryToken;
        if (poolCoreInfo.poolType == PoolType.EXTERNAL) {
            primaryToken = ETH;
        } else {
            primaryToken = bdefiCore().primaryTokens(poolCoreInfo.token);
        }
        _addToHookFullfilmentQueue(
            poolCoreInfo.parentToken, primaryToken, parentAmount -
parentDistributed, true, false, hook, leaderboard
        );

        // rest of tokens go to token leaderboard
        _addToHookFullfilmentQueue(
            poolCoreInfo.token, poolCoreInfo.token, tokenAmount -
tokenDistributed, true, false, hook, leaderboard
        );
    }
```

We can clearly see that for token add to fulfillment is called which increases the queue for the bdefi token in hooks contract

```
 function _addToHookFullfilmentQueue(
        address token,
        address receiver,
        uint256 amount,
        bool isForLeaderboard,
        bool isForGenesis,
        IBDeFiHook hook,
        IBDeFiLeaderboard leaderboard
    ) internal {
        if (token == ETH) {
            _processEthFullfilment(receiver, amount, isForLeaderboard,
isForGenesis, leaderboard);
        } else if (token == WETH_ADDRESS) {
            IWETH(WETH_ADDRESS).withdraw(amount);
            _processEthFullfilment(receiver, amount, isForLeaderboard,
isForGenesis, leaderboard);
        } else {
            IERC20(token).safeTransfer(address(hook), amount);
  ===>          hook.addToFulfillmentQueue(token, receiver, amount,
isForLeaderboard, isForGenesis);
        }
    }
```

So calling collect trading fees in before swap function before any of the internal logic is invoked helps in more fuflfillment of the tokens.

Recommendations

In beforeSwap function add one line

```
 function _beforeSwap(address, PoolKey calldata poolKey, SwapParams
calldata params, bytes calldata)
        internal
        override
        returns (bytes4, BeforeSwapDelta, uint24)
    {
        // TruncatedGeoOracle tracking
        _beforeSwap(poolKey);
        PoolId poolId = poolKey.toId();

        PoolCoreInfo memory poolCoreInfo =

IBDeFiLpManager(bdefiCore.getContract(BDeFiContractNames.LP_MANAGER)).getP
oolCoreInfo(poolId);

   +++
IBDeFiLpManager(bdefiCore.getContract(BDeFiContractNames.LP_MANAGER)).coll
ectPoolTradingFees(poolId);

        if (!fullyInitialized[poolCoreInfo.token]) revert
```

```
BDeFiHook__TokenNotFullyInitialized();
        uint256 lookbackTime =
_bdefiConfig().getPoolLookbackConfig(poolId);
        lastTradedTs[poolCoreInfo.token] = block.timestamp;

        // if pool observations are missing, skip fulfillment
        if (HookLibrary.isObservationMissing(poolKey, lookbackTime)) {
            return (this.beforeSwap.selector,
BeforeSwapDeltaLibrary.ZERO_DELTA, 0);
        }

        // Apply buy/sell tax if needed
        int256 specifiedDelta;
        int256 unspecifiedDelta;

        specifiedDelta = _applyBuySellTax(params, poolKey,
BalanceDelta.wrap(0), true);

        // If user is buying the token, try to fulfill from collected fees
first
        if (poolCoreInfo.isParentToken0 == params.zeroForOne) {
            if (params.amountSpecified < 0) {
                uint160 twapSqrtPriceX96 =
HookLibrary.getTwapSqrtPriceX96(poolKey, uint32(lookbackTime));
                (specifiedDelta, unspecifiedDelta) =
_processInternalTokenFulfillment(
                    specifiedDelta, unspecifiedDelta,
params.amountSpecified, twapSqrtPriceX96, poolCoreInfo
                );
            }
        }

        return (this.beforeSwap.selector,
toBeforeSwapDelta(int128(specifiedDelta), int128(unspecifiedDelta)), 0);
    }
```

[L-06] Config setters accross the protocol accept arbitrary values and admin misconfiguration will cause permanent protocol DoS

Description

`BDeFiConfig.sol#setUint16ConfigVariable()` and `BDeFiConfig.sol#setUint256ConfigVariable()` accept any value with no bounds validation. Six distinct parameters, if set incorrectly, cause permanent and irreversible DoS across critical protocol functions:

`POINTS_MULTIPLIER_DIVIDER = 0` - `BDeFiPoints.sol#buyMultiplier()` computes `pointsAmount * PRECISION / multiplierDivider`. Division by zero reverts permanently. The entire

multiplier purchase system is bricked.

`FEE_QUEUE_PROCESS_LIMIT = 0` - `BDeFiHook.sol#processExternalToken()` processes `min(queueLength, limit) = 0` items per call. `totalAmountFulfilled` stays `0`. The strict equality check `totalAmountFulfilled != fulfilmentAmount` always reverts for any non-zero amount. External fee distribution is permanently DoS'd.

`LEADERBOARD_PAYOUT_BPS > 10000` - `BDeFiLeaderboard.sol#finalizeEpoch()` computes `totalPayoutAmount = _applyBps(balance, payoutBps) > balance`. The subsequent `tokenPoolBalances[token] -= totalPayoutAmount` underflows and reverts. Epoch finalization is permanently broken for every token.

`MIN_LAUNCH_DURATION > MAX_LAUNCH_DURATION` - `BDeFiConfig.sol#_validateLaunchDuration()` requires `duration >= MIN && duration <= MAX`. If MIN > MAX, no valid duration exists and every launch attempt reverts. The entire token launch system is DoS'd.

`BPS_VOID_LAUNCH_FEE > 10000` - `BDeFiLaunch.sol#voidLaunch()` computes `voidFee = _applyBps(ethParticipation, voidFeeBps) > ethParticipation`. `_safeEthTransfer(leaderboard, voidFee)` fails because the contract doesn't hold that much ETH. `info.status = VOIDED` is never reached. All failed launches are permanently stuck in `OPEN` status with their ETH locked.

`BPS_REFERRAL_DISCOUNT or BPS_REFERRAL_FEE > 10000` - `BDeFiPayments.sol#_calculateReferralAmounts()` computes `totalPayment = amount - _applyBps(amount, discount)` where `_applyBps(amount, 15000) > amount` → underflow revert. Alternatively `genesisAmount = totalPayment - referrerAmount` where `referrerAmount > totalPayment` -> underflow. Every single `processPayment` call fails. Launch fees, CTO fees, whitelist fees, boost fees - the entire payment system is bricked.

Recommendations

Add bounds validation to each setter call or more robustly add a dedicated validation function that checks all dependent invariants after any config update:

```
function setUint16ConfigVariable(bytes32 nameHash, uint16 value) public
onlyAdmin {
    if (nameHash == BDeFiConfigNames.LEADERBOARD_PAYOUT_BPS && value >
BPS_BASE)
        revert BDeFiConfig__InvalidValue();
    if (nameHash == BDeFiConfigNames.BPS_VOID_LAUNCH_FEE && value >
BPS_BASE)
        revert BDeFiConfig__InvalidValue();
    if (nameHash == BDeFiConfigNames.BPS_REFERRAL_DISCOUNT && value >=
BPS_BASE)
        revert BDeFiConfig__InvalidValue();
    if (nameHash == BDeFiConfigNames.BPS_REFERRAL_FEE && value >=
BPS_BASE)
        revert BDeFiConfig__InvalidValue();
    uint16Values[nameHash] = value;
```

```
        emit ConfigUpdated(nameHash, value);
    }
```

Apply equivalent guards to `setUint256ConfigVariable` for `POINTS_MULTIPLIER_DIVIDER` (must be > 0), `FEE_QUEUE_PROCESS_LIMIT` (must be > 0) and the duration pair (must maintain MIN ≤ MAX after update).

[L-07] `setBlueprint` unconditionally increments version so duplicate calls shift all future CREATE2 addresses

Description

`BDeFiCloner.sol#setBlueprint()` runs `blueprintVersions[name]++` on every call regardless of whether the blueprint address actually changed. If admin accidentally calls it twice with the same address, the version bumps unnecessarily. All future module deployments for that blueprint type use the new version in their CREATE2 salt, producing different addresses than expected.

Recommendations

Only bump the version when the address actually changes:

```
if (blueprintRegistry[name] != blueprint) {
    blueprintVersions[name]++;
}
blueprintRegistry[name] = blueprint;
```

[L-08] Removing whitelisted external parent permanently breaks swap routes for all child tokens

Description

`BDeFiSwapHandler.sol#buildAndStoreTokenRoutes()` is called once at launch and never again. For tokens launched with an external parent (e.g., USDC as primary LP), the full ETH -> USDC -> token route is stored in `SwapHandler`. When that external parent is later removed from the whitelist, the stored routes become stale. Every subsequent call to `BDeFiSwapHandler.sol#swapEthToInternalToken()` for affected child tokens fails at the Uniswap level. No `rebuildRoutes` function exists and the only fix will be a contract upgrade.

Recommendations

Add an admin-accessible route rebuild function to `BDeFiSwapHandler.sol` that re-runs route construction for a specific token, callable after whitelist changes.

[L-09] `_liquidityActionV4` uses `block.timestamp` for currency1 permit2 approval instead of `deadline`

Description

In `BDeFiLpManagerActions.sol#_liquidityActionV4()`, the Permit2 approval for `currency0` uses `uint32(deadline)` while `currency1` uses `uint32(block.timestamp)`. The `currency1` approval expires at the current block rather than at the operator-specified deadline. Since `modifyLiquidities` is called in the same transaction no funds are currently at risk, but the inconsistency is fragile against future restructuring.

Recommendations

```
PERMIT2.approve(
    Currency.unwrap(poolKey.currency1),
    address(posManagerV4),
    uint160(args.amount1),
    uint32(deadline) // was: uint32(block.timestamp)
);
```

[L-10] Epoch 0 can never be finalized - first two weeks of leaderboard earnings permanently locked

Description

`BDeFiLeaderboard.sol#finalizeEpoch()` reverts when `finalizedEpochs[token] == currentEpoch`. Both values default to `0`. During the entire first `EPOCH_LENGTH` (2 weeks), `CommonUpgradeable.sol#getCurrentEpoch()` returns `0`, so every finalization attempt hits `0 == 0` and permanently reverts. All fees accumulated during epoch 0 are unclaimable for that window.

This is further compounded if `GENESIS_TIMESTAMP` is set to a future value at deployment - `getCurrentEpoch()` returns `0` for the entire period until genesis is reached, extending the broken window indefinitely. `BDeFiCore.sol#initialize()` only checks `genesisTimestamp != 0`, so a far-future value passes validation with no on-chain recovery path.

Recommendations

For the epoch 0 issue, treat the default `0` as "never finalized" instead of "finalized at epoch 0":

```
if (finalizedEpochs[token] != 0 && finalizedEpochs[token] == currentEpoch)
{
```

```
        revert BDeFiLeaderboard__EpochIncomplete();
    }
```

For the genesis timestamp issue, add an upper-bound check in `BDeFiCore.sol#initialize()`:

```
if (genesisTimestamp == 0 || genesisTimestamp > block.timestamp) revert
Common__ZeroValue();
```

[L-11] The `uint64` parameter in `setPointsPack` prevents admin from restoring default pack sizes

Description

`BDeFiPoints.sol#setPointsPack()` takes `uint64 pointsAmount`. The default packs set in `initialize()` use `11_000 ether`, `36_000 ether` and `150_000 ether` - all far exceeding `uint64` max (~18.45 ether in 18-decimal units). If any default pack is deleted via `deletePointsPack()`, it can never be restored through the public admin interface. The internal `_setPointsPack()` correctly accepts `uint256`, making this a pure type mismatch on the public function signature.

Recommendations

Change the parameter type to match the internal function.

[L-12] Stablecoin depeg risk due to hardcoded 1:1 conversion

Description

`BDeFiPayments._convertUsdPeggedToken` assumes that any token marked as `USD_PEGGED` is always worth exactly $1 per token (according to its decimals). It multiplies the USD amount (2 decimals) by the token's decimals and divides by 100 (10^2). This relies on the token maintaining a perfect peg indefinitely.

```
function _convertUsdPeggedToken(uint256 usdAmount, address paymentToken)
internal view returns (uint256) {
    return (usdAmount * (10 ** IERC20Metadata(paymentToken).decimals())) /
(10 ** USD_DECIMALS);
}
```

If a stablecoin depegs (e.g., USDC depegs to $0.90), the protocol will still treat it as $1. Users could exploit this by paying fees with the depegged token at a discount, causing the protocol to receive less value than intended. Conversely, if the token trades above $1, users would overpay, but that is less likely to be exploited.

Recommendations

- Introduce an oracle for stablecoins (e.g., Chainlink price feeds) to obtain the current USD price of the token, and use that for conversion.
- If that is not feasible, clearly document the assumption and accept the risk. Consider adding a circuit breaker or minimum/maximum deviation checks.

[L-13] LIFO fee queue processing leads to permanent fee starvation

Description

In `BDeFiHook.sol`, the hook keeps an array-based queue of pending fees (`_tokenFulfillmentQueue`). This is done by pushing the latest fees into the array in `addToFulfillmentQueue` and `_addToFulfillmentQueue`

```
            _tokenFulfillmentQueue[token].push(
                FeesQueueItem({receiver: receiver, isLeaderboard:
isForLeaderboard, amount: amount})
            );
```

When fulfilling these fees, both `_internalFulfillFromQueue` and `_externalFulfillFromQueue` iterate by popping elements off the end of the array.

```
        FeesQueueItem storage item = queue[queue.length - 1];
        uint256 take = item.amount < remainingToFulfill ? item.amount :
remainingToFulfill;
        // ... transfers ...
        if (take == item.amount) {
            queue.pop();
        } else {
            item.amount -= take;
        }
```

By reading from `queue.length - 1` and calling `.pop()`, the queue operates as a LIFO (Last-In-First-Out) stack. The newest fees added to the system are fulfilled first. If the protocol generates fees faster than the `queueProcessLimit` can handle, the oldest fees at index `0` may never be reached, permanently locking those users' funds at the bottom of the stack.

Recommendations

Implement a true FIFO (First-In-First-Out) queue using a mapping structure with `head` and `tail` pointers. This allows processing and delete the oldest entries first without having to shift array elements.

[L-14] Creator can receive additional support allocation, exceeding intended creator cap

Description

In `BDeFiConfig.validateLaunchOptions`, after validating ETH allocations and enabling additional support if `addSupport > 0`, the function `_validateAdditionalSupport` is called. This function iterates over `opts.supportReceivers`, unpacking each entry into an address and bps. It checks for duplicate addresses and ensures the total bps sums to `BPS_BASE`, but it **does not** verify that any of the addresses is different from the token creator (the `account` parameter passed to `validateLaunchOptions`). Consequently, a creator can include their own address among the support receivers and receive a portion of the ETH that is meant for additional support.

```
        function _validateAdditionalSupport(TokenLaunchOptions calldata
opts) internal {
            uint256 numSupport = opts.supportReceivers.length;
            // ...
            for (uint256 i = 0; i < numSupport; i++) {
                (address addr, uint16 bps) =
opts.supportReceivers[i].toAddressAlloc();
                // ... checks for zero, duplicates, total bps
                totalBps += bps;
            }
            // ...
        }
```

This allows the creator to circumvent the per-module caps and potentially receive more ETH than intended. For example, if the creator allocation is limited to 8%, the creator could add themselves as a support receiver with another 5%, effectively receiving 13% of the mint proceeds.

Recommendations

Add a check in `_validateAdditionalSupport` (or after it) to ensure that none of the support receiver addresses equals the creator (`account`).

[L-15] Missing operator reimbursement in `distributeTaxes` may stall tax distribution

Description

The `BDeFiTaxDistributor.sol` contract contains the `distributeTaxes` function, which is exclusively callable by an operator:

```
    function distributeTaxes(
        address token,
        uint256 amount,
        uint256 minAmountOutV4,
        uint256 minAmountOutV3,
        uint256 deadline
    ) external onlyOperator {
        ...
```

Unlike other operator-controlled functions in the protocol (e.g., `initializeLp`, `buyBurn`, `processExternalToken`), this function completely lacks an implementation for gas reimbursement.

Because operators are designed to be compensated for their execution costs, the absence of a reimbursement mechanism means off-chain automated operators will systematically avoid calling this function, causing tax funds to accumulate and protocol operations to stall.

Recommendations

Add a `reimbursement` parameter to `distributeTaxes` and implement standard operator reimbursement tracking logic.

[I-01] Hardcoded mainnet addresses in `Common.sol` will break multichain deployments

Description

The `Common.sol` contract hardcodes several addresses corresponding to Ethereum Mainnet for core dependencies, including Permit2, Uniswap V4 Pool Manager, Uniswap V3 Factory/Router, and WETH.

```
    IAllowanceTransfer constant PERMIT2 =
IAllowanceTransfer(0x000000000022D473030F116dDEE9F6B43aC78BA3);
    IPoolManager constant UNI_V4_POOL_MAN =
IPoolManager(0x000000000004444c5dc75cB358380D2e3dE08A90);
    IUniswapV3Factory constant UNI_V3_FACTORY =
IUniswapV3Factory(0x1F98431c8aD98523631AE4a59f267346ea31F984);
    address constant WETH_ADDRESS =
0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    address constant UNI_V3_ROUTER =
0xE592427A0AEce92De3Edee1F18E0157C05861564;
    address constant UNI_UNIVERSAL_ROUTER =
0x66a9893cC07D91D95644AEDD05D03f95e1dBA8Af;
```

However, the protocol is explicitly designed to be deployed on multiple chains, as evidenced by `LpManagerLibrary.getInitialCardinality()`, which contains logic for Optimism, Base, Arbitrum, Avalanche, BNB Smart Chain, and Polygon. The addresses for `WETH` and other dependencies are completely

different on these networks. Deploying this code on any chain other than Ethereum Mainnet will result in immediate integration failures and locked funds.

Recommendations

Refactor the architecture to accept these dependency addresses via the constructor/initializer rather than hardcoding them as constants, or use a chain-specific configuration registry.

[I-02] `MAX_ETH_PRICE_STALENESS` default value is high for ETH/USD price feed

Description

In `BDeFiPayments.sol`, the protocol utilizes an ETH/USD Chainlink price feed to calculate USD fees and conversions. The staleness check compares the `updatedAt` timestamp against a configurable `MAX_ETH_PRICE_STALENESS`:

```
        uint256 staleness = block.timestamp - updatedAt;
        if (staleness >
config.uint256Values(BDeFiConfigNames.MAX_ETH_PRICE_STALENESS)) {
            revert BDeFiPayments__StaleChainlinkAnswer(staleness);
        }
```

In `BDeFiConfig.sol`, the default value for this staleness is set to 1 day:

```
        uint256Values[BDeFiConfigNames.MAX_ETH_PRICE_STALENESS] = 1 days;
```

The Chainlink ETH/USD feed on mainnet has a heartbeat of 1 hour (3600 seconds). By allowing a 1-day staleness, the protocol exposes itself to accepting prices that are up to 23 hours stale if the feed stops updating. This can lead to severely incorrect fee calculations during periods of high volatility or network congestion.

Recommendations

Change the default configuration of `MAX_ETH_PRICE_STALENESS` to accurately reflect 1 hour.

[I-03] Typo: `hanlder` instead of `handler`

Description

In `BDeFiTaxDistributor._processTax`, the variable `hanlder` is misspelled.

```
```

```
        IBDeFiSwapHandler hanlder =
IBDeFiSwapHandler(bdefiCore().getContract(BDeFiContractNames.SWAP_HANDLER)
);
        token.safeIncreaseAllowance(address(hanlder), taxAmount);
```

Recommendations

Correct the spelling to `handler`.